Lembar Kerja Praktikum P07 – Exception Handling

Lembar kerja ini menggantikan laporan praktikum. Isi **sambil** mengerjakan setiap langkah di modul, lalu kumpulkan di akhir sesi sebagai `LK-P07-<NIM>.pdf` bersama zip kode Anda.

Petunjuk pengisian

- Jawab singkat dan jelas. Satu sampai tiga kalimat per pertanyaan sudah cukup.
- Tempel output program sebagai **teks**, bukan tangkapan layar.
- Di L2 dan L3, isi kolom **prediksi** sebelum menjalankan program. Prediksi yang salah tidak mengurangi nilai; yang dinilai adalah penjelasan selisihnya.
- Total nilai LK 50 poin. Bagian L0 wajib diisi, tanpa poin.

L0 – Identitas dan parameter (wajib)

Isian	Jawaban
Nama	Gina Ramadhani
NIM	2511533014
Kelas	B
Tanggal praktikum	09 November 2026
Versi Java (baris pertama `java -version`)	25
Saldo awal e-wallet (`NimParams`)	Rp250000
Stok awal KOPI-001	3
Stok awal RDG-002	2

L1 – Membaca crash dan menangani input (8 poin)

(a) Bedah stack trace `CrashDemo` skenario 2 dan 3 (4 poin). Contoh pengisian untuk skenario 1 ada di modul.

Yang dicari	Skenario 2	Skenario 3
Jenis exception	NumberFormatException	NullPointerException
Checked atau unchecked?	Unchecked	Unchecked

Yang dicari	Skenario 2	Skenario 3
Pesan	For input string: "dua"	annot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null
Baris kode Anda yang pertama (file:baris, method)	CrashDemo.java:28, method readQuantity	CrashDemo.java:32, method shippingCity
Dipanggil dari (file:baris, method)	CrashDemo.java:43, method main	CrashDemo.java:44, method main
Apa penyebabnya, dengan kata-kata Anda sendiri?	Input jumlah berupa teks "dua", sehingga tidak dapat dikonversi menjadi bilangan bulat menggunakan Integer.parseInt().	Objek Address bernilai null, tetapi program tetap mencoba memanggil method getCity() dari objek tersebut.

(b) Jumlah `-2` (2 poin). Sebelum `ShopApp` diperbaiki, apa yang terjadi saat Anda memilih produk 2 lalu mengetik jumlah `-2`? Mengapa tidak muncul `NumberFormatException`? Apa beda "format input salah" dengan "aturan bisnis dilanggar"?

Jawaban:

Nomor produk di luar katalog dicegah menggunakan if karena input nomor produk sudah berupa angka, tetapi nilainya tidak sesuai dengan jumlah produk yang tersedia. Kondisi tersebut merupakan pelanggaran aturan bisnis, bukan kesalahan format input. Sementara itu, teks yang bukan angka ditangkap menggunakan catch NumberFormatException karena terjadi kesalahan saat program mencoba mengubah teks menjadi bilangan bulat menggunakan Integer.parseInt().

(c) `if` atau `catch`? (2 poin). Mengapa nomor produk di luar katalog dicegah dengan `if`, sedangkan teks yang bukan angka ditangkap dengan `catch (NumberFormatException _)`?

Jawaban:

- `if` digunakan untuk memeriksa apakah nilai input memenuhi aturan program.
- `catch` digunakan untuk menangani exception yang terjadi ketika proses program dijalankan.

L2 – Prediksi dulu, baru jalankan (12 poin)

Isi kolom **Prediksi** sebelum menjalankan program. Tulis baris-baris output sesuai urutan, misalnya `A1, A2, A4, A5`.

`FlowDemo` bagian A, B, C (7 poin: A 2, B 3, C 2)

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
A: `partA("12")` lalu `partA("12rb")`	• `partA("12")`: A1 muncul, angka berhasil dibaca, A2 menampilkan jumlah 12, kemudian A4 dan A5 dijalankan. • `partA("12rb")`: A1 muncul, tetapi proses konversi angka gagal. Program masuk ke catch A3, lalu tetap menjalankan A4 dan A5.	A1 mulai A2 jumlah = 12 A4 finally A5 lanjut A1 mulai A3 bukan angka A4 finally A5 lanjut	Ya	Prediksi sesuai karena input pertama berhasil diubah menjadi angka, sedangkan input kedua menghasilkan `NumberFormatException`. Ketika exception terjadi, program menjalankan catch, kemudian finally tetap dijalankan.

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
B: `partB(300000, "3")` lalu `partB(300000, "0")`	• `partB(300000, "3")`: B1 dan B2 dijalankan karena pembagian berhasil. Setelah itu B4 dijalankan, kemudian hasil pembagian ditampilkan oleh main. • `partB(300000, "0")`: B1 menampilkan nilai bulan 0, tetapi proses pembagian mengalami kesalahan. B4 tetap dijalankan, lalu exception diteruskan ke main dan ditangkap oleh catch bagian luar. Baris B5 kedua dan B6 diperkirakan tidak dijalankan.	B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 B1 bulan = 0 B4 finally B7 main menangkap: / by zero	Ya	Pada pemanggilan pertama pembagian berhasil. Pada pemanggilan kedua terjadi `ArithmeticException` karena pembagian dengan nol. Catch di `partB` tidak menangani exception tersebut, sehingga exception diteruskan ke main. Akibatnya, B5 kedua dan B6 tidak dijalankan.
C: `partC(5)`	`partC(5)`: C1 dijalankan terlebih dahulu. Setelah itu, meskipun sudah ada perintah return, Bagiannya tetap melewati finally C2 sebelum nilai sisa stok ditampilkan oleh main.	C1 cek stok C2 finally C3 sisa = 4	Ya	Walaupun method memiliki perintah return di dalam try, finally tetap dijalankan terlebih dahulu sebelum nilai dikembalikan. Karena stok awal 5 dikurangi 1, hasil akhirnya adalah 4.

`CariProdukDemo` bagian D (3 poin)

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`... CariProdukDemo 2`	Sepertinya nomor 2 akan mengambil produk kedua dari daftar SKU. Setelah itu, bagian finally tetap dijalankan.	D1 SKU: RDG-002 D5 pencarian selesai	Tidak ada catch yang dijalankan karena nomor 2 merupakan indeks produk yang valid. Program langsung menampilkan SKU produk, kemudian menjalankan blok `finally` (D5).
`... CariProdukDemo abc`	Input tidak bisa diubah menjadi angka, jadi kemungkinan masuk ke catch yang menangani format angka. Finally tetap dijalankan.	D2 nomor harus bilangan bulat D5 pencarian selesai	Catch `NumberFormatException` (D2) dijalankan karena input `abc` tidak dapat dikonversi menjadi bilangan bulat. Catch `IllegalArgumentException` tidak dijalankan karena Java memilih catch pertama yang cocok. Setelah itu, finally (D5) tetap dijalankan.
`... CariProdukDemo 8`	Nomor produk melebihi jumlah data yang tersedia, sehingga diperkirakan masuk ke catch indeks array. Finally tetap dijalankan	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	Catch `ArrayIndexOutOfBoundsException` (D3) dijalankan karena nomor 8 berada di luar indeks array yang tersedia. Setelah exception ditangkap, finally (D5) tetap dijalankan.
`... CariProdukDemo` (tanpa argumen)	Karena tidak ada nomor yang diberikan, program akan melempar kesalahan argumen dan masuk ke catch yang sesuai. Finally tetap dijalankan.	D4 Berikan satu nomor produk. D5 pencarian selesai	Catch `IllegalArgumentException` (D4) dijalankan karena tidak ada argumen nomor produk yang diberikan. Exception tersebut ditangkap oleh catch D4, kemudian finally (D5) tetap dijalankan.

L2.c – Ubah urutan catch (2 poin). Tempel pesan error compiler setelah catch `IllegalArgumentException` dipindah ke paling atas, lalu jelaskan artinya dalam satu kalimat.

Pesan error:

```
Unreachable catch block for NumberFormatException
```

Lokasi: `CariProdukDemo.java`, baris 16, kolom 18.

Artinya:

Compiler menolak susunan catch tersebut karena *NumberFormatException* merupakan turunan dari *IllegalArgumentException*. Ketika catch induknya diletakkan lebih dahulu, exception turunannya sudah ikut tercakup sehingga catch *NumberFormatException* menjadi tidak dapat dijangkau.

L3 – Custom exception dan checkout (10 poin)

(a) Error berantai (2 poin). Setelah `PaymentMethod.pay` diubah menjadi `void ... throws PaymentFailedException`, berapa error yang muncul, dan di file mana saja? Mengapa satu perubahan di interface memengaruhi file-file itu?

Jumlah error: **4** File: **`BankTransferPayment.java`, `CreditCardPayment.java`, `EWalletPayment.java`, `CheckoutService.java`**

Alasan:

Perubahan method `pay()` pada interface `PaymentMethod` dari method yang mengembalikan nilai menjadi `void` menyebabkan kelas-kelas yang mengimplementasikannya harus menyesuaikan tipe pengembalian method tersebut. Akibatnya, tiga kelas pembayaran mengalami ketidaksesuaian tipe pengembalian. Selain itu, `CheckoutService` masih mencoba mengembalikan hasil dari `payment.pay()`, padahal method tersebut sekarang tidak mengembalikan nilai (`void`). Oleh karena itu, satu perubahan pada interface dapat memengaruhi seluruh kelas yang mengimplementasikan atau menggunakan method tersebut.

(b) Enam skenario `ScenarioRunner` (4 poin). Isi kolom prediksi dulu (pakai parameter L0), baru jalankan. Hasil diisi salah satu dari `[OK]`, `[SALDO]`, `[BAYAR]`, `[STOK]`, `[INVALID]`.

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S1 Kopi + Rendang, Reguler, e-wallet	[OK]	2 / 1 / 5 / Rp35.000	[OK]	2 / 1 / 5 / Rp35.000	Ya

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S2 Kopi x3, Reguler, transfer bank	[STOK]	2 / 1 / 5 / Rp35.000	[STOK]	2 / 1 / 5 / Rp35.000	Ya
S3 Rendang x2, Reguler, e-wallet	[STOK]	2 / 1 / 5 / Rp35.000	[STOK]	2 / 1 / 5 / Rp35.000	Ya
S4 Kopi, Ekspres, kartu kredit	[SALDO]	2 / 1 / 5 / Rp35.000	[SALDO]	2 / 1 / 5 / Rp35.000	Ya
S5 Keranjang kosong	[INVALID]	2 / 1 / 5 / Rp35.000	[INVALID]	2 / 1 / 5 / Rp35.000	Ya
S6 Teh x2, Reguler, e-wallet	[SALDO]	2 / 1 / 5 / Rp35.000	[SALDO]	2 / 1 / 5 / Rp35.000	Ya

Pilih satu skenario yang prediksinya meleset (atau, jika semua cocok, skenario yang paling menarik), lalu jelaskan mengapa hasilnya seperti itu. Sebut juga mengapa pesan `[STOK]` menyebut "diminta 1".

Jawaban:

Skenario yang paling menarik adalah S2 karena pembelian 3 kopi menghasilkan [STOK], padahal pesan error menyebutkan diminta 1 dan tersedia 0. Hal ini terjadi karena program melakukan reservasi stok satu per satu untuk setiap produk. Setelah dua kopi berhasil direservasi, stok menjadi 0 sehingga reservasi kopi ketiga gagal. Pesan tersebut menyebutkan diminta 1 karena setiap pemanggilan reserve() hanya memproses satu produk, bukan seluruh jumlah dalam keranjang.

(c) Mengembalikan stok lalu `throw e` (2 poin). Di method `checkout`, mengapa blok catch mengembalikan stok lalu melempar ulang exception yang sama, bukan mencetak pesan untuk pembeli? Apa yang terjadi pada stok jika `release` dihapus?

Jawaban:

Blok catch mengembalikan stok menggunakan release() agar stok yang sudah direservasi tidak hilang ketika pembayaran gagal. Setelah itu, throw e digunakan untuk melempar kembali exception yang sama agar dapat ditangani oleh pemanggil, seperti ShopApp.buy(), sehingga pesan kegagalan dapat disampaikan kepada pembeli. Jika release() dihapus, stok akan terus berkurang meskipun transaksi gagal sehingga jumlah stok menjadi tidak sesuai dengan kondisi sebenarnya.

(d) Urutan catch di `buy` (2 poin). Mengapa catch `InsufficientBalanceException` harus ditulis di atas catch `PaymentFailedException`? Apa yang terjadi jika urutannya ditukar?

Jawaban:

InsufficientBalanceException harus ditulis sebelum PaymentFailedException karena InsufficientBalanceException merupakan turunan dari PaymentFailedException. Jika urutannya ditukar, catch InsufficientBalanceException tidak dapat dijalankan karena exception tersebut sudah ditangkap oleh catch induknya. Akibatnya, Java akan memberikan compile error karena catch yang lebih spesifik menjadi tidak terjangkau.

L4 – Katalog dari file (6 poin)

(a) Baris katalog yang dilewati (3 poin).

Baris ke-	Isi baris	Apa masalahnya?	Exception yang tertangkap (nama class)
3	KUE-003;Kue Lapis Legit;95rb;800	Harga 95rb tidak dapat dikonversi menjadi angka.	`IllegalArgumentException`
5	BATIK-005;Kain Batik Tulis	Jumlah kolom kurang, hanya ditemukan 2 kolom dari 4 kolom yang diperlukan.	`IllegalArgumentException`
6	SAMB-006;Sambal Lado Mudo 200g;-45000;250	Harga produk bernilai negatif.	`IllegalArgumentException`

(b) Ketika file tidak ada (3 poin). Tempel output `java -cp bin shop.app.ShopApp katalog-cadangan.csv`. Apa isi `e.getMessage()` dan `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang dilihat pengguna, dan mengapa itu kurang membantu?

Output:

```
Toko tidak bisa dibuka: Katalog gagal dibaca: katalog-cadangan.csv
Penyebab asli: java.nio.file.NoSuchFileException: katalog-cadangan.csv
```

Jawaban:

Isi e.getMessage() adalah Katalog gagal dibaca: katalog-cadangan.csv, sedangkan e.getCause() berisi exception asli, yaitu NoSuchFileException karena file katalog tidak ditemukan. Jika ShopApp hanya mencetak e.getCause().getMessage(), pengguna hanya akan melihat nama file katalog-cadangan.csv. Hal tersebut kurang membantu karena tidak menjelaskan penyebab kegagalan secara lengkap. Oleh karena itu, e.getMessage() lebih informatif karena memberikan konteks bahwa toko tidak dapat membaca katalog.

L5 – Code review FragileShop (10 poin)

Tabel temuan (8 poin, 2 per method).

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`payOrder`	Exception PaymentFailedException ditangkap tetapi tidak ditangani atau ditampilkan.	Kegagalan pembayaran hilang tanpa diketahui pembeli maupun programmer.	Hapus try-catch dan tambahkan throws PaymentFailedException agar kegagalan diteruskan ke pemanggil.
`isInStock`	Menggunakan reserve() dan release() hanya untuk memeriksa stok.	Pengecekan stok menimbulkan efek samping karena mengubah stok sementara.	Gunakan inventory.available(sku) > 0 untuk memeriksa stok secara langsung.
`readCatalogHeader`	Menggunakan null sebagai penyebab pada CatalogLoadException.	Penyebab asli kesalahan membaca file hilang sehingga programmer sulit mencari sumber masalah.	Ganti null dengan e pada new CatalogLoadException("Katalog gagal dibaca: " + file, e).
`addToCart`	Kesalahan input jumlah dan stok habis ditangani dengan catch yang sama serta pesan yang sama.	Pembeli tidak mendapatkan informasi yang jelas apakah input salah atau stok tidak mencukupi.	Pisahkan penanganan NumberFormatException dan OutOfStockException dengan pesan yang sesuai.

Output `FragileShop` setelah `payOrder` dan `readCatalogHeader` diperbaiki (2 poin).

```
Output:
1) Pembayaran gagal: Saldo tidak cukup: perlu Rp85000, tersedia Rp50000
2) Kopi tersedia? true
3) Gagal: For input string: "dua"
   Gagal: Stok KOPI-001 tidak cukup: diminta 5, tersedia 1
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: java.nio.file.NoSuchFileException: katalog-lama.csv
```

Penutup

Hasil SelfCheck terakhir. Salin dua baris berikut dari output SelfCheck.

```
=== HASIL: ...
=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
```

=== TANTANGAN: ...

Tidak ada, karena bagian Tantangan dinonaktifkan pada fitur SelfCheck, sehingga kolom tersebut tidak ditampilkan.

Refleksi (4 poin).

1. (2 poin) Mengapa kesalahan ketik pembeli (misalnya `dua` sebagai jumlah) perlu ditangani berbeda dari stok yang habis? Hubungkan dengan pilihan checked/unchecked.

Jawaban:

Kesalahan input ditangani dengan unchecked exception, sedangkan stok habis ditangani dengan checked exception karena keduanya memiliki penyebab berbeda.

2. (1 poin) Apakah pesan exception teknis, misalnya `For input string: "dua"`, sebaiknya ditampilkan langsung kepada pembeli? Beri satu contoh dari praktikum ini.

Jawaban:

Tidak. Contohnya, `For input string: "dua"` diubah menjadi "Jumlah barang harus berupa angka."

3. (1 poin, dinilai penuh asal diisi jujur) Satu hal tentang exception yang masih membingungkan atau ingin Anda tanyakan di pertemuan berikutnya.

Jawaban:

Saya masih bingung menentukan kapan harus menggunakan checked dan unchecked exception.